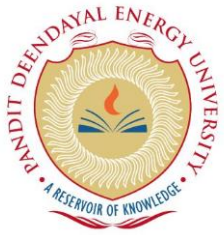
MedVerify: Camera-Verified Smart Pill Box for Elderly Medication Adherence

IoT Project Report

Submitted in Partial Fulfilment of the requirements
for
IoT laboratory course
of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING
by

ISHAAN GOSWAMI
(23BCP290)

Under the Guidance of
PROF. _____



Department of Computer Science & Engineering,
School of Technology,
Pandit Deendayal Energy University, Gandhinagar 382 426
Nov 2026

Certificate of Originality of Work

I hereby declare that the B. Tech. IoT Project entitled "MedVerify: Camera-Verified Smart Pill Box for Elderly Medication Adherence" submitted by me for the partial fulfilment of the IoT laboratory course of Bachelor of Technology to the Dept. of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me under the supervision of Prof. _____.

I also declare that this written submission adheres to the university guidelines for originality and includes proper citations and references wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Signature of the Student
Ishaan Goswami
23BCP290
Place: Gandhinagar
Date: _____

Signature of the Supervisor
Name of the Supervisor
Designation of the Supervisor

Acknowledgement

We express our sincere gratitude to our supervisor for guidance and encouragement throughout this project, and to the Department of Computer Science & Engineering, School of Technology, PDEU, for the laboratory facilities. We thank the laboratory staff for their support with the hardware, and our classmates and families for their help in testing the system.

(Ishaan Goswami)

INDEX

Topic No.	Title	Page No.
	Certificate of Originality of Work and Acknowledgement	2
	Index	3
1	Introduction	4
1.1	Background	4
1.2	Motivation	4
1.3	Objectives	4
1.4	Problem Statement	4
1.5	Approach	4
1.6	Scope of the Project	5
1.7	Organization of the Rest of the Report	5
2	Literature Review	5
2.1	Previous Approaches to Solve the Problem	5
3	Hardware Design	5
4	Software Design	6
5	Results and Discussion	7
6	Conclusions and Future Scope (including ESP32 Wi-Fi plan)	7
	References	8
	Appendix: Commands and Pin Map	9

1. Introduction

1.1 Background

Elderly patients often take several medicines a day and forget doses, take a dose twice, or take the wrong medicine. Family caretakers usually live or work elsewhere and have no reliable way to know what happened. Commercial smart pill boxes mostly record that a lid was opened, which says nothing about whether the right medicine, or the right number of pills, was taken out. MedVerify is an IoT pill box that combines a reminder device (ESP32 with one LED per compartment and a buzzer) with a camera that watches the compartments continuously and counts the medicine in each one, and web applications for the patient, the caretaker and an administrator.

1.2 Motivation

Medication non-adherence is common among older adults and is a major cause of avoidable hospital admissions [1], [2]. A device that only beeps cannot tell a missed dose from a taken one. Using a camera turns the box into a verifier: it can see that one pill left compartment A at dose time, or that two pills were taken instead of one, and alert immediately, both at the box and on the caretaker's phone.

1.3 Objectives

- Remind the patient at dose time by lighting the correct compartment and sounding a buzzer.
- Verify, with a camera, how many pills were removed from which compartment, without any sensor inside the box.
- Detect and alert on wrong compartment, too many pills, several compartments at once, partial or missed doses, pills put back, and low stock.
- Give the patient a simple bilingual (English/Hindi) screen and the caretaker a password-protected web app.
- Run reliably on low-cost hardware: an ESP32, a USB webcam and a laptop.

1.4 Problem Statement

Design and build a low-cost IoT system that reminds an elderly patient to take medicine from a specific compartment and verifies, without contact sensors, whether the correct number of pills was removed from the correct compartment, reporting the outcome to the patient and to a remote caretaker in real time. The system must not raise false alarms when hands move over the box, pills are rearranged, or room light changes. Note: a camera can confirm that medicine was REMOVED; it cannot confirm that it was swallowed, and the system never claims ingestion.

1.5 Approach

The compartments are four squares drawn with a pen on a white sheet (a 2x2 grid); coloured paper pieces stand in for pills during testing. A webcam above the sheet finds the drawn squares automatically, learns what each compartment looks like empty and loaded, and from then on measures the area covered by medicine in each square. Dividing this area by the area of one pill (learned from the declared count at calibration) gives a pill count that does not depend on where the pills lie. A Flask server runs the dose schedule, drives the ESP32 over USB serial, and serves the three web applications. Every decision waits until the scene has been still for a moment, so moving hands are never mistaken for removals.

1.6 Scope of the Project

In scope: four compartments, scheduled and manual doses, pill counting by camera, alerts on the device, the patient screen and the caretaker app, and complete control from one command line (`mv`). Out of scope for this iteration: verifying ingestion, stacked pills (they hide each other), white pills on white paper (low contrast), and wireless operation (planned, Chapter 6).

1.7 Organization of the Rest of the Report

Chapter 2 reviews existing approaches. Chapters 3 and 4 describe the hardware and software design. Chapter 5 presents the tests and results, and Chapter 6 concludes with future scope, including the ESP32 Wi-Fi design.

2. Literature Review

2.1 Previous Approaches to Solve the Problem

Approach	How it works	Limitation addressed by MedVerify
Alarm pill boxes / phone apps	Timer beeps or notifies at dose time	No evidence the dose was taken; patient must self-report
Sensor pill bottles/boxes [3]	Logs when a lid opens or a bottle is moved	Opening is not taking; cannot tell how many pills or which pill
Load-cell (weight) boxes	Weight drop per compartment	Needs one sensor per compartment; small pills weigh < 1 g, close to sensor noise
Automatic dispensers [4]	Motorised release of one dose	Expensive, refill-heavy, mechanically complex
Vision-based adherence [5]	Camera/ML observes pills or the act of taking them	Often needs training data or a phone held by the patient; MedVerify uses fixed overhead camera and classical image processing with no training

Table 2.1: Comparison of existing approaches

MedVerify sits between lid-sensor boxes and full vision systems: one inexpensive camera covers all compartments with no wiring inside the box, and classical OpenCV processing [6] (no model training) keeps it explainable and runnable on any laptop.

3. Hardware Design

The hardware has two parts: the indicator unit (ESP32 DevKit, four LEDs, buzzer) and the observation unit (USB webcam above the compartment sheet). Both connect to a laptop that runs the server. The ESP32 was chosen for its low cost, 3.3 V GPIOs that drive LEDs directly, built-in USB-serial (CP210x) for today's wired link, and integrated Wi-Fi/Bluetooth for the wireless version (Chapter 6) [7].

Component	Connection	Purpose
ESP32 DevKit (ESP32-D0WD-V3)	USB to laptop (CP210x, 115200 baud)	Receives commands, drives LEDs and buzzer
LED A / B / C / D + 220-330 ohm	GPIO 16 / 17 / 18 / 19	Lights the compartment whose dose is due
Buzzer module	Signal GPIO 25, VCC 3V3, GND	Reminder beeps, ALARM siren, WARN beeps

Component	Connection	Purpose
On-board LED	GPIO 2	Boot and command-received indication
USB webcam (640x480)	USB to laptop	Watches the four drawn compartments
Compartment sheet	White paper, 2x2 pen grid, taped down	Defines compartments A-D

Table 3.1: Hardware components and connections

Firmware (Arduino C++, flashed from the command line with arduino-cli, no IDE) accepts line commands: A-D (light that LED and beep three times), STOP, TEST, DIAG (12 s wiring check), ALARM (3 s two-tone siren for too many pills), WARN (two slow beeps for an advisory) and HELP. Each command replies with a fixed text, so the server knows the box is alive; a missing reply raises a device-error alert while the dose continues on screen.

4. Software Design

The software is written in Python (Flask, OpenCV, NumPy) with HTML/JavaScript front ends. Fig. 4.1 shows the data flow.

Webcam -> Camera service (detect squares, count pills, hand guard) -> heartbeat every 1-2 s -> Flask server (schedule, dose manager, alerts) -> ESP32 over USB serial (LEDs, buzzer) | Patient screen, Admin console, Caretaker app (browser, polling)

Fig. 4.1: System data flow

4.1 Vision pipeline

- **Compartment detection:** A black-hat filter keeps thin dark pen lines; line runs at tilts up to 15 degrees are kept, gaps bridged. Each region of paper fully enclosed by lines is a compartment; its exact outline (not a shrunken box) is stored, so pills anywhere inside it count.
- **Counting:** medicine = pixels whose colour (Lab chroma > 14) or brightness (> 28) differs from the EMPTY photo, after evening out overall light; count = medicine area / area of one pill.
- **Confirmation:** a new count is accepted only after 2 agreeing observations over 2 s on a still scene; a count may rise only when the estimate is within 0.2 of a whole number (a false rise would hide a dose).
- **Hand guard:** an object lying on both sides of a compartment's pen line is a hand or sleeve reaching in: that compartment is 'view blocked' and its count is held.
- **Robustness:** frames are judged against the calibration sharpness; a small slide of the sheet (up to 40 px) is followed automatically.

4.2 Dose manager and alerts

A dose opens on schedule (or manually), lights the LED and starts a 30-minute window with reminders every 5 minutes. A dose is taken when the compartment's count drops by the dose size.

Alert	Trigger	Response
DOSE_REMOVED	Count drops by the dose size	LED off; patient screen 'Done'; caretaker informed
TOO_MANY (critical)	More pills than the dose	ALARM siren; red 'Stop, put the extra back'; caretaker critical alert
WRONG_COMPARTMENT	Pills taken from another box during a dose	Screen names the right box; LED re-points and beeps

Alert	Trigger	Response
MULTI_COMPARTMENT	Two or more boxes within 20 s	'Take one medicine at a time'; WARN beeps
PARTIAL / MISSED	Fewer pills / nothing by the deadline	Caretaker warned; dose closed as partial or missed
PUT_BACK	Count rises soon after a dose	Caretaker warned that the dose may not be taken
UNSCHEDULED / LOW_STOCK	Removal with no dose / few pills left	Caretaker informed

Table 4.1: Alerts generated by the dose manager

4.3 Applications

- **Patient screen:** full-screen, large text in English and Hindi: which box, how many pills, Stop-sound button, alert banners with OK.
- **Admin console:** live compartment counts, camera status, doses, schedule, device controls.
- **Caretaker app:** password-protected; dose history, alerts, send a reminder, edit the schedule, recalibrate.
- **Control:** one command-line tool runs everything (start/stop, camera set-up, box tests, firmware upload, doses, tests); `live_demo.bat` starts a demonstration in one step.

5. Results and Discussion

Test	Result	Notes
Automated unit/integration tests	257 pass	Synthetic and real photos of the drawn grid, dose logic, alerts
Drawn-square detection, real photos	4/4 squares	Crooked sheet (7 deg), faint lines must be darkened
Live count accuracy (4,4,4,3 pieces)	4.03, 4.02, 3.97, 3.02	Estimates within 0.05 of truth on a still scene
Hand resting over a box	Pass	Box marked view blocked; no false removal
Rearranging / putting back pills	Pass	Count unchanged; put-back flagged
No false alarm in 20 s untouched	Pass	Guided demo S2
ESP32 LEDs A-D, ALARM, WARN	Pass	Confirmed on the real board
Counted dose during live run (S3, S5, S6)	Failed in one run	Traced to the camera/sheet moving after calibration; every box read 'view blocked'

Table 5.1: Test results

On a fixed camera and taped sheet the counts were stable to within a few percent and every guard against false alarms worked. The main practical lesson is that calibration is tied to the physical position of the camera and sheet: when either moved (for example while wiring the hardware), all compartments appeared blocked and dose events stopped. Remedies applied or planned: tape the sheet and fix the camera rigidly, re-run the three-step set-up after any move, and show an explicit 'camera moved' warning (Chapter 6). Crumpled paper pieces change visible area by up to 25 % when moved, so flat pieces of uniform size give the most reliable counts.

6. Conclusions and Future Scope

MedVerify shows that a single overhead camera and classical image processing can verify, compartment by compartment, how many pills were removed at dose time, and that combining this with an ESP32

indicator gives immediate, specific feedback (right box, too many, one at a time) at the box, on a bilingual patient screen and on the caretaker's app. The system runs on inexpensive hardware, needs no sensor inside the box, and avoids false alarms from hands, rearranging and lighting.

6.1 ESP32 Wi-Fi: justification and implementation plan

Today the ESP32 is tethered to the laptop by USB, which limits where the box can stand. The ESP32 has 802.11 b/g/n Wi-Fi built in [7], so the same board becomes a wireless node without extra hardware, which is the main reason it was chosen over boards such as the Arduino Uno.

- **Connectivity:** the ESP32 joins the home Wi-Fi (credentials set once through a Wi-Fi-manager captive portal) and keeps a connection to the server.
- **Protocol:** the server already contains an ESP32 HTTP adapter (`device_mode: esp32`); the firmware will expose the same commands (A-D, STOP, ALARM, WARN) as HTTP endpoints, or subscribe to an MQTT topic [8] for lower latency and automatic reconnection.
- **Health:** the ESP32 sends a heartbeat every few seconds; missing heartbeats raise the same device-error alert used today.
- **Security:** a shared token on every command, and HTTPS/TLS for the public caretaker link.
- **Camera on the edge:** with an ESP32-CAM module the camera could also move onto the box, removing the laptop for detection and making the device self-contained.

6.2 Other future work

- Warning on screen when every compartment is blocked for long (camera or sheet moved), with one-click re-calibration.
- Caretaker access from outside the home through a secure tunnel, and SMS/WhatsApp/push notifications.
- Pill identification by colour and shape, physical trays with fixed markers, and a battery-backed enclosure.
- Adherence reports for doctors and a clinical trial with real patients.

References

[1] L. Osterberg and T. Blaschke, "Adherence to medication," N. Engl. J. Med., vol. 353, no. 5, pp. 487-497, Aug. 2005.

[2] World Health Organization, "Adherence to long-term therapies: evidence for action," WHO, Geneva, Switzerland, 2003.

[3] AdhereTech, "Smart pill bottle for medication adherence," AdhereTech Inc., New York, NY.

[4] Hero Health, "Hero smart medication dispenser," Hero Health Inc., New York, NY.

[5] AiCure, "Artificial intelligence platform for medication adherence," AiCure LLC, New York, NY, 2020.

[6] G. Bradski, "The OpenCV library," Dr. Dobb's J. Software Tools, vol. 25, no. 11, pp. 120-125, 2000.

[7] ESP32 Series Datasheet, v4.x, Espressif Systems, Shanghai, China, 2023.

[8] MQTT Version 5.0, OASIS Standard, Mar. 2019.

Appendix: Commands and Pin Map

Command	Purpose
live_demo.bat	Start server, camera and pill box; open the three apps
mv cam regions preview / auto	Find the drawn squares / save them as compartments A-D
mv cam empty	Empty-sheet reference photo
mv cam loaded A=4 B=4 C=4 D=3	Loaded photo with pill counts; starts monitoring
mv dose start A pills=1	Start a dose now (LED A, buzzer, patient screen)
mv box test | alarm | warn	Check LEDs and buzzer patterns
mv status	Counts, camera, device and open doses

Pins: LED A GPIO16, LED B GPIO17, LED C GPIO18, LED D GPIO19, buzzer GPIO25, status LED GPIO2.